



基於 MQTT 及 Node-RED 物聯網伺服器於溫度監控系統的運用 Application of MQTT and Node-RED IoT Server in Temperature monitoring System

曾羿嘉¹
YiJia-Tseng

沈志雄²
Chih-Hsiung Shen

陳永欽³
Yeong-Chin Chen

摘要

最近物聯網技術越來越普及，智慧家居系統中的溫度控制系統也變得越來越受歡迎。這項研究旨在開發一種基於雲端物聯網的 PID 控制器，以實現智慧家居系統中的高效溫度控制。研究中將使用 MQTT 協議實現物聯網設備之間的通信，我們發現 MQTT 比傳統的傳輸方式更加可靠與快速，所以我們以 MQTT 為系統中樞進行資料的傳輸，使用者介面上使用 Node-RED 來建立一個網頁界面，以顯示和控制溫度控制系統，Node-RED 可以使用各式的節點組成我們想要的功能並在網頁中顯示，我們選用一個溫控箱來模擬居家溫控的環境，並使用 PID 控制算法對溫度進行控制。控制器的運行將在 Linux 伺服器上實現，並通過 PWM 及微功率開關實現控制訊號的輸出，以控制 Arduino 控制板上的風扇和燈泡。

Node-RED 流程將與 MQTT 協議集成，以實現與物聯網設備的通信並發送控制訊號。總體而言，這個系統將提供一個用戶友好和高效的智慧家居溫度控制解決方案。

關鍵詞：雲端物聯網、PID 控制器、MQTT、Node-RED

Abstract

Recently, the Internet of Things (IoT) technology has become more and more popular, and temperature control systems in smart home systems are also becoming increasingly popular. This study aims to develop a cloud-based PID controller using IoT technology to achieve efficient temperature control in smart homes. The study will use the MQTT protocol to enable communication between IoT devices. We found that MQTT is more reliable and faster than traditional transmission methods, so we chose MQTT as the central hub for data transmission. Node-RED will be used to create a web interface to display and control the temperature control system. Node-RED can use various nodes to create the desired function and display it in the webpage. We have selected a temperature control box to simulate the home temperature control environment and use the PID control algorithm to control the temperature. The controller will run on a Linux server and control signals will be output using PWM and low-power switches to control fans and light bulbs on the Arduino control board.

¹彰化師範大學機電系碩士生 amos8655@gmail.com

²彰化師範大學機電系教授兼系主任 hilbert@cc.ncue.edu.tw

³亞洲大學資工系教授 ycchenster@gmail.com

The Node-RED flow will be integrated with the MQTT protocol to enable communication with IoT devices and transmit control signals. Overall, this system will provide a user-friendly and efficient smart home temperature control solution.

Keywords: IoT, PID Controller, MQTT, Node-RED

1.前言

近幾年智能家居的應用(Dong, B,2021)越來越多元，目前發展開關控制及自動化情境控制逐漸成熟，這些電器及設備透過自動化控制而大幅降低能源損耗，但我們認為若只是情境控制無法達到能源運用最佳化，如果再透過智能學習的演算法，利用感測器蒐集環境反饋數據學習，可以更細緻調控電力輸出，可以讓整體能源效率更進一步提升。

目前科技發達，智能控制系統應用如恆溫控制系統、空調系統、燈光控制系統、平衡系統等相關應用相當廣泛，遍及軍事、工業、農業、食品、汽車、檢測領域。這些控制系統可能透過感測器監控，可以減少電氣設備損耗、風險及能源損耗，藉由數據蒐集資料庫分析，進一步大數據分析可預警設備異常，預估設備維護時機，降低設備維護成本與提升產品良率，大幅降低設備異常所造成的災害損失。

本計畫研究動機旨在開發一種與行動裝置結合的雲端控制系統，以作為恆溫控制與即時溫度冷卻系統之用途，以廣泛容易取得的 Arduino 模組為基礎，便宜低廉的 ESP8266 無線網路傳輸模組，固態電子繼電器、溫度感測數據製作出智慧插座，透過 ESP8266 用 Wi-Fi 模組傳輸到雲端 MQTT 伺服器，後端為控制器 PID 控制演算法，控制以燈泡與風扇模擬的環境，於網頁(Node-RED)上可以讓使用者透過瀏覽器看到即時的數據變化圖，利於控制環境溫度。

2.研究原理

2.1 PID 控制器

PID (Proportional Integral Derivative)(Aghaei, V,2015)控制系統是一種自動控制系統，它的目的是通過控制信號來消除目標值和實際值之間的誤差。PID 控制器的輸入為誤差信號，表示目標值和實際值之間的差異。當誤差存在時，PID 控制器會生成一個適當的控制信號來消除誤差。

為了提高系統的響應速度和減少誤差，PID 控制器使用了三個增益參數：比例增益、積分增益和微分增益。比例增益用於調節控制信號輸出，以消除誤差。積分增益用於消除偏移誤差，也就是長期存在的誤差，但它會引起過衝現象，即控制信號的輸出會超過目標值。為了減少這種現象，使用了微分增益做設計。

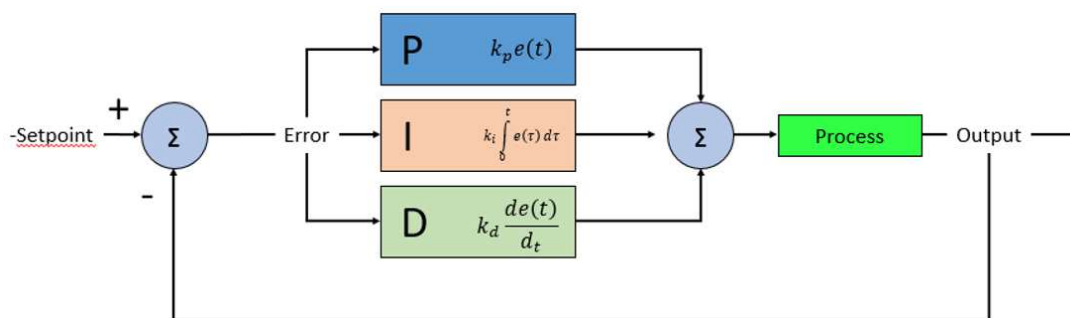


圖 1.PID 流程圖

PID 控制器是一種基於比例、積分和微分糾正算法的控制器，它將這三種算法的

加權和作為其輸出，以實現更加精確的控制。其輸入為誤差值，即設定值減去測量值後的結果，或是由誤差值衍生的信號。比例增益控制誤差值的大小，積分增益消除積累的誤差，微分增益則控制當前的誤差變化率。通過這三種糾正算法的組合，PID 控制器可以應對不同的控制系統需求，實現更加穩定和精確的控制。PID 演算法可以用式一表示：

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{d}{dt} e(t) \quad (1)$$

上式中參數：

K_p = 比例增益，調適參數

K_i = 積分增益，調適參數

K_d = 微分增益，調適參數

e = 誤差=設定值(SP)- 回授值(PV)

t = 目前時間

τ = 積分變數，數值從 0 到目前時間

在 PID 控制系統中，比例增益是根據誤差信號直接生成控制信號的一個參數，它的值越大，控制信號輸出對誤差的響應越強。積分增益則根據誤差信號的積分值生成控制信號的一個參數，它的值越大，對於長期存在的誤差越敏感。微分增益則是根據誤差信號的微分值生成控制信號的一個參數，它的值越大，對誤差變化的快慢越敏感。

綜合以上三種增益，PID 控制器能夠快速且穩定地消除誤差，從而實現自動控制的目的。該系統的運算流程如圖所示，首先計算誤差信號，然後通過比例增益、積分增益和微分增益生成控制信號，最終將控制信號輸出到控制系統中，實現自動調節的目的。PID 的轉移函數控制器如式二：

$$G_{PID}(s) = K_p + \frac{K_i}{s} + k_d s \quad (2)$$

2.1 脈衝寬度調變零交越控制

脈衝寬度調變(Pulse Width Modulation, PWM)(Li, Z,2012)，是一種調節電子設備輸出訊號的技術。在 PWM 技術中，輸出訊號的頻率固定，而訊號的高低電平則依據脈衝寬度的不同而變化。脈衝寬度是指一個週期中，高電平的時間長度，以週期的百分比表示，通常稱為「佔空比」(duty cycle)。

脈衝寬度調變(PWM)是一種將類比信號轉換為數位脈波的技術，轉換後的脈波週期固定。本系統中的升溫和降溫裝置外接電源使用了 110V 的交流電。要使用 PWM 調節輸出電壓，需要考慮交流電正弦波形。在調節時，需要在每半個週期的前一個零交越點到後一個零交越點之間，使用 PWM 原理調節電壓開關比例，即佔空比。假設佔空比為 60%，將半波分成 128 個等份，每一等份時間約為 65 微秒(μs)。我們使用的零交越點偵測電路如圖所示。其中 AC 電源經過橋式整流器驅動 4N25 光耦合器，當電壓輸出為零時輸入 Arduino，即表示偵測到 AC 電源的零交越點。Arduino 隨即輸出 PWM 訊號控制 AC 電源供應給升溫或降溫裝置，從而完成 PWM 功率控制功能。

$$\text{佔空比} = \frac{\text{電信號不為 0 的時間}}{\text{電信號為 0 的時間} + \text{電信號不為 0 的時間}} \times 100\%$$

圖 2.佔空比計算方式

本系統中的燈泡和風扇都是由交流電源供電。以燈泡為例，它使用 110V 的交流電供電。一個完整的交流電波形包括正半波和負半波，而與 0V 相交的點稱為零交越點。在控制交流電輸出電壓時，正半波和負半波都以零交越點為基準。將半波分成 128 等份，每一等份為 65 微秒。然後使用 PWM 調整導通比例。假設佔空比為 50%，則調光器在第 1 至第 64 段不導通(延遲觸發)，從第 65 段開始觸發直到下一次零交越關閉。每調高或降低一段，延遲時間就會減少或增加 65 微秒。

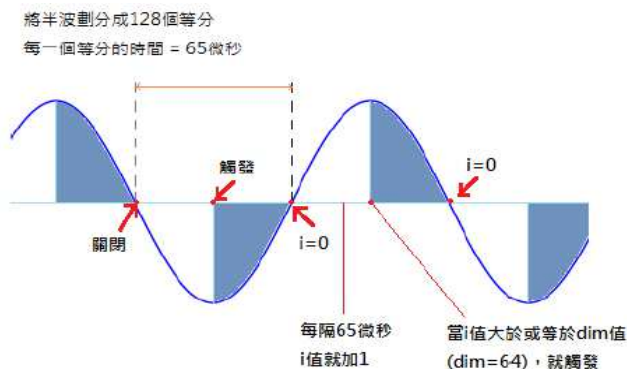


圖 3.零交越示意圖

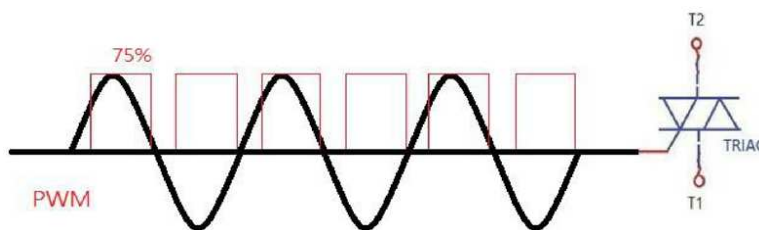


圖 4.佔空比 75%控制 TRIAC 示意圖

3.系統實作

3.1 系統流程圖

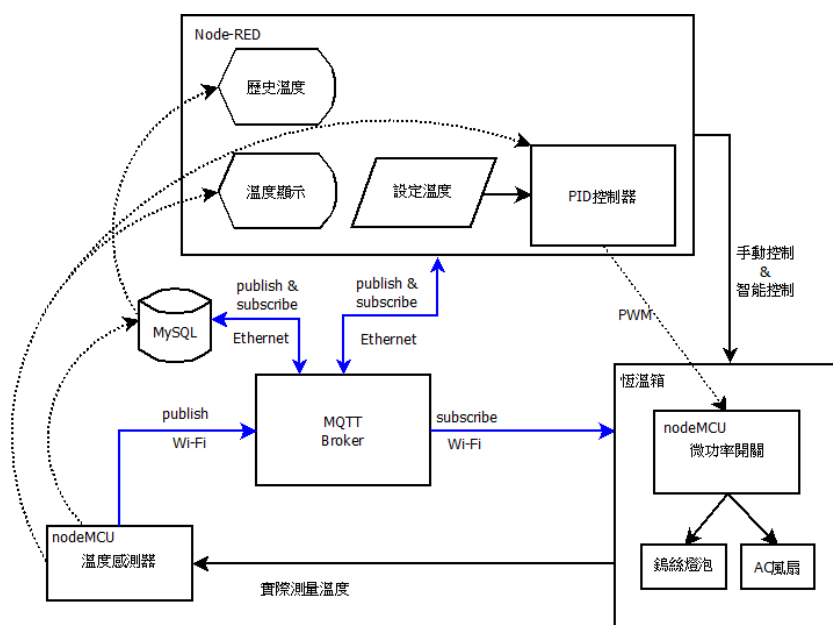


圖 5.系統流程圖

在這個系統中我們使用兩顆 NodeMCU 做溫度接收與功率接收的控制，以及兩顆 Arduino Pro Mini 做鎢絲燈泡與 AC 燈泡的 PWM 控制，Node-R 做溫度與各項功率數值的顯示與控制，所有的資訊如溫度、功率控制等數值都會優先發布至 MQTT 伺服器再讓各個設備訂閱取得。

3.2 交流電控制電路設計

3.2.1 智慧插座

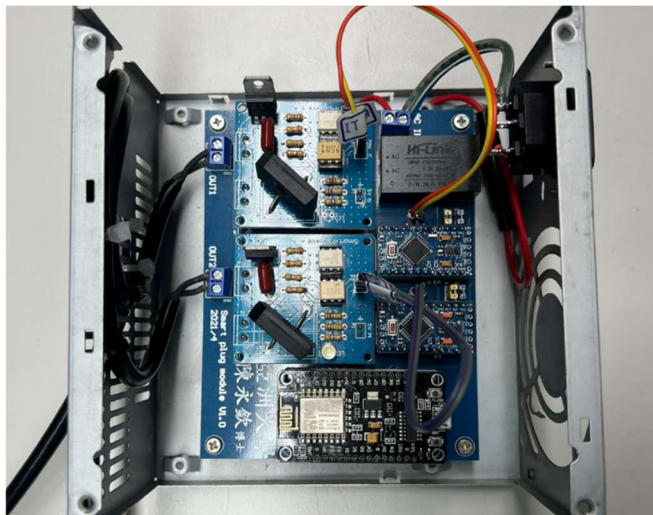


圖 6. 插座本體

插座本體由橋式整流器、110V to 5V 的變壓器、兩顆 Arduino pro mini 和一顆 NodeMCU 組成。當插座接收到輸入電流時，電流會先經過橋式整流器和變壓器，再由 NodeMCU 和 arduino pro mini 進行進一步的控制。

3.2.2 NodeMCU

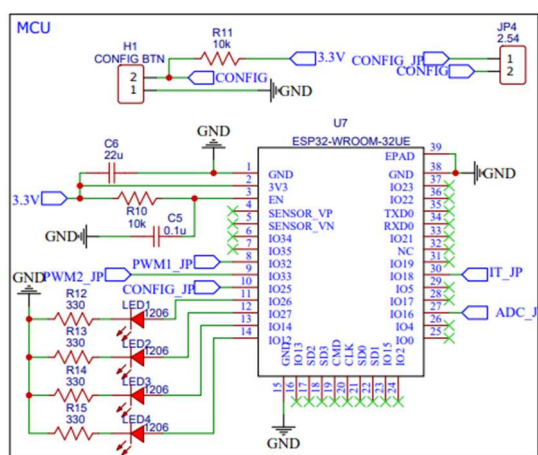


圖 7. NodeMCU 電路圖

我們使用 NodeMCU(Parihar, Y. S,2019)進行溫度的監控與功率的控制，在溫度監控上我們使用 MLX90614 連結 NodeMCU 將監測到的溫度傳輸至 MQTT 上，並由 Node RED 取得溫度後進行進一步控制。

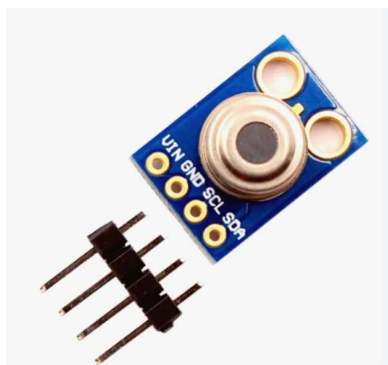


圖 8. 溫度感測器 MLX90614

MLX90614(Jin, G,2019)是一種紅外非接觸溫度計。它的 TO-39 金屬封裝中集成了紅外感應熱電堆探測器晶片和信號處理專用集成晶片。由於集成了低噪聲放大器、17 位模數轉換器和強大的數字信號處理單元，因此能夠實現高精度和高分辨率的溫度計。

該溫度計出廠時已經校準過，具有數字 SMBus 輸出，可在整個溫度範圍內完全訪問測量到的溫度，分辨率為 0.02°C 。用戶可以將數字輸出配置為脈衝寬度調製 (PWM)。

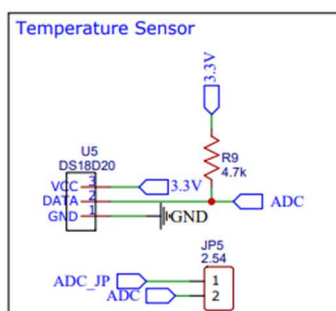


圖 9. 溫度感測器電路圖

在功率控制上我們從 MQTT 上取得控制數值後將數值傳輸給 arduino pro mini 進行控制。

3.2.3 Arduino pro mini

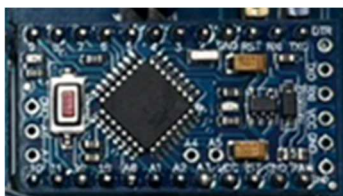


圖 10. arduino pro mini 本體

我們使用 Arduino pro mini(Rusimamto, P,2021)來取得 NodeMCU 讀取 MQTT 伺服器上的功率數值，1 為風扇 2 為燈泡，以此來控制橋式整流的電壓輸出。

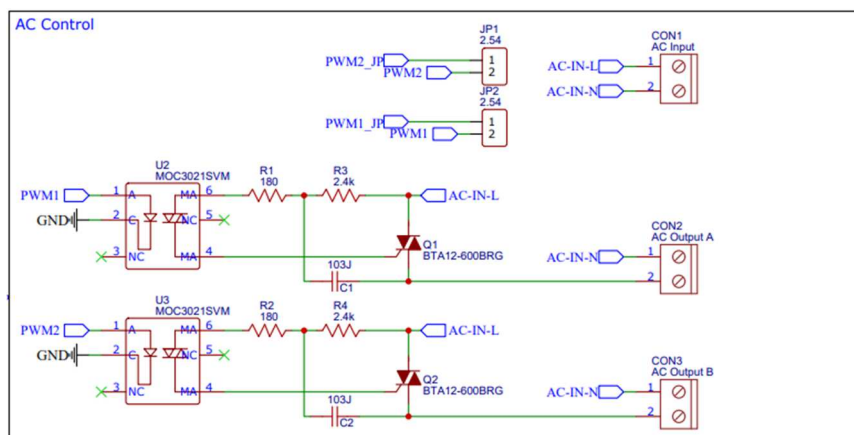


圖 11. PWM 控制電路圖

本系統中的燈泡和風扇都是由交流電源供電。以燈泡為例，它使用 110V 的交流電供電。一個完整的交流電波形包括正半波和負半波，而與 0V 相交的點稱為零交越點。在控制交流電輸出電壓時，正半波和負半波都以零交越點為基準。將半波分成 128 等份，每一等份為 65 微秒。然後使用 PWM 調整導通比例。假設佔空比為 50%，則調光器在第 1 至第 64 段不導通(延遲觸發)，從第 65 段開始觸發直到下一次零交越關閉。每調高或降低一段，延遲時間就會減少或增加 65 微秒(如圖)。

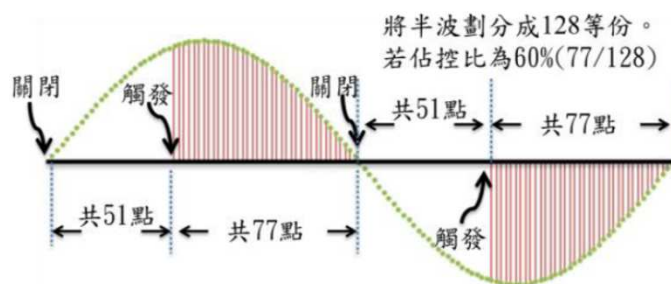


圖 12. PWM 佔空比示意圖

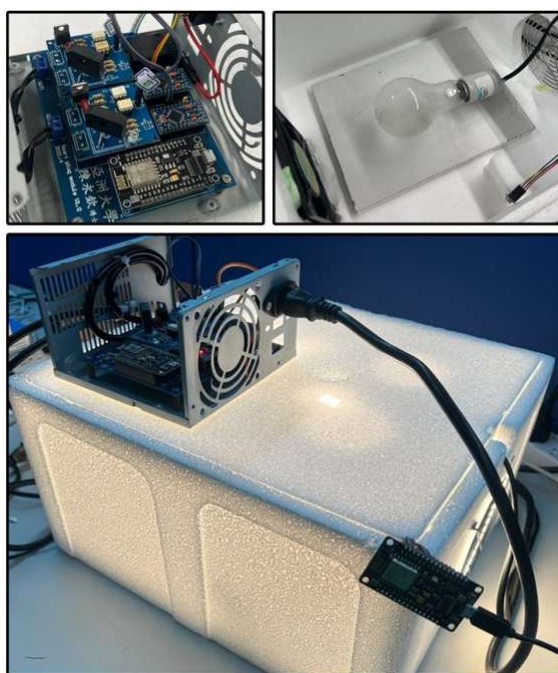


圖 13.溫控箱內部以及運作

3.3 MQTT

MQTT (Message Queuing Telemetry Transport)(Hunkeler, U,2008)是一種適合物聯網通訊的輕量級協定。使用 MQTT 的系統通常包含客戶端(Client)、發布者(Publisher)和 MQTT Broker(伺服器)。系統的運作方式是，客戶端與 MQTT Broker 建立連接，並訂閱感興趣的主題。發布者向 MQTT Broker 發送消息，消息中帶有主題和內容。MQTT Broker 接收到消息後，會根據主題轉發給所有訂閱該主題的客戶端。客戶端接收到消息後，會進行相應的處理。客戶端也可以向 MQTT Broker 發送指令，例如訂閱、取消訂閱和發布消息等。如果客戶端與 MQTT Broker 斷開連接，Broker 會將其訂閱的主題下線。

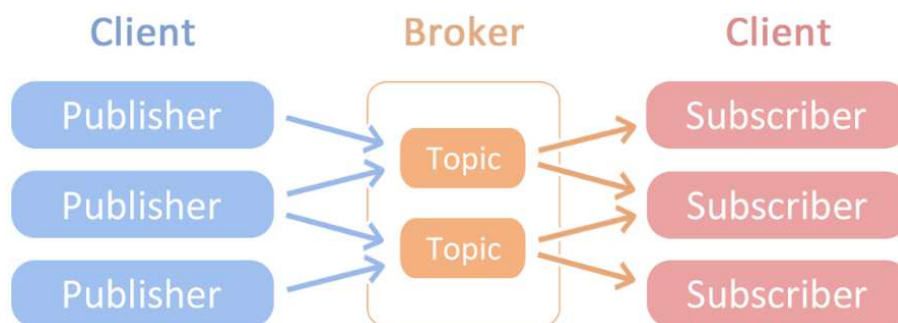


圖 14.MQTT 運作圖

MQTT 的輕量、可靠和靈活性，適用於物聯網領域中的聯網裝置之間的通訊。MQTT 協定的優點在於通訊開銷小，資源需求較低，支援消息有效期限和遺囑消息，且具有 QoS 等完整的消息保證機制。同時，MQTT 協定的開發文檔詳細，並有多種不同的開源軟體實現，因此實現 MQTT 的客戶端和 Broker 相對容易。由於 MQTT 是一種應用層協定，其具有靈活的特性，可以實現不同的使用場景，被廣泛應用於物聯網領域。因此我們選用 MQTT 而不使用 http 來做為我們的傳輸工具。

3.4 NodeRed 建置網頁

Node-RED(Kodali, R,2018)是一款基於 Node.js(Tilkov, S,2010)的開源流程自動化工具，使用簡潔易用的編輯器，Node-RED 也有很多的節點庫(插件)，我們使用了 Dashboard 他是一個可視化工具，可以幫助用戶快速構建基於 Web 的應用程序。使用 Node-RED Dashboard，我們可以輕鬆地創建自己的儀表板，監控傳感器數據，控制裝置，顯示數據報表等等。

他提供了各種 UI 元素，例如按鈕、文本框、滑動條、圖表等等，可以根據需要自由組合。用戶可以通過拖放的方式快速構建儀表板，也可以使用 JavaScript 編寫自定義 UI 元素和行為。

Node-RED 也支持與多種後端數據庫和 API 進行交互，例如 MongoDB、MySQL、PostgreSQL、MQTT、RESTful API 等等。用戶可以輕鬆地從這些數據源中提取數據並顯示在儀表板上。

在我們的專案中我們使用到 Dashboard 的 text box、chart 跟 gauge。

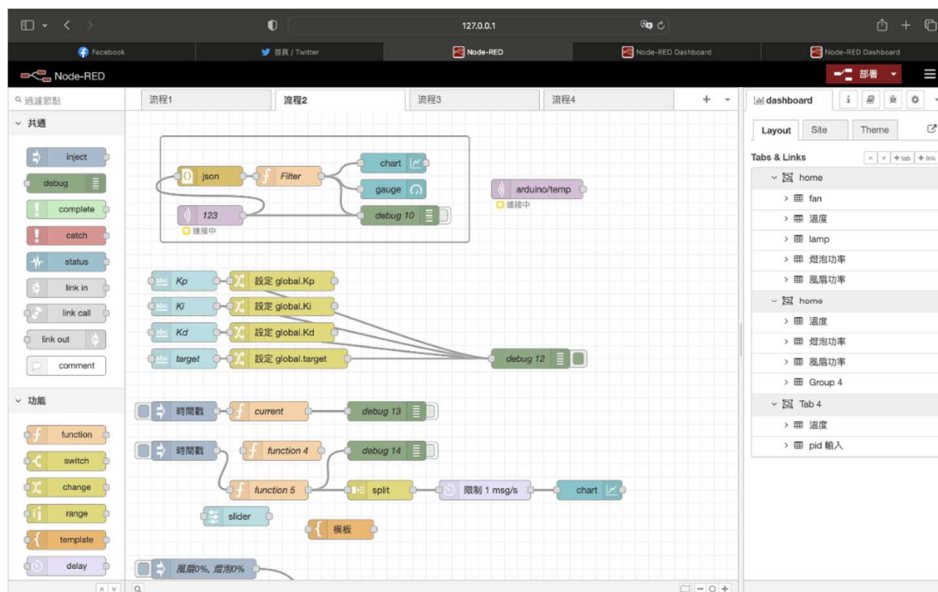


圖 15. NodeRed 節點介面

text box 使用在輸入 K_p 、 K_i 、 K_d 的調整上，chart 跟 gauge 用於監測溫度和功率，使用 Dashboard 可以很輕鬆的製作出圖形介面，並讓使用者一目了然現在的狀況。

在紅框中可以讓使用者自行輸入 K_p 、 K_i 、 K_d 三個數值，也可以輸入溫度數值進行控制，在綠框中我們可以看到繪製出的各項數值，也可以看到現在的溫度方便使用者隨時監控。

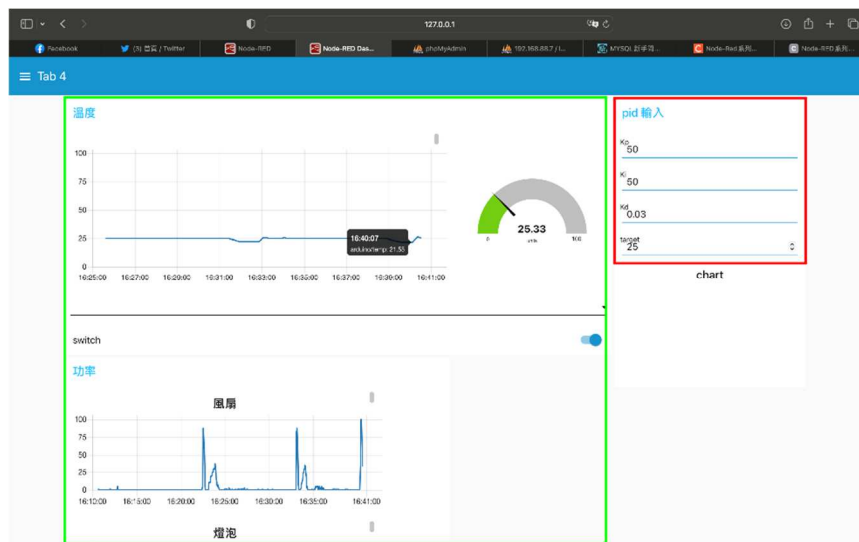


圖 16.使用者介面

3.5 PID 控制

這段程式碼使用了比例積分微分(PID)控制器來進行控制，以維持系統中目標溫度的穩定。

在誤差(目標溫度與當前溫度之差)大於或等於 5 度時，系統將直接根據誤差的正負值調節燈泡或風扇的功率，從而達到升溫或降溫的效果。

當誤差小於 5 度時，則使用 PID 控制器來進一步控制溫度。具體而言，系統首

先將當前誤差透過比例，積分和微分三個分量相加，得到一個控制輸出。然後，根據誤差正負值調整燈泡或風扇的功率，逐漸調節溫度，最終穩定於目標溫度值附近。

在此過程中，比例分量用於調整系統的響應速度，積分分量用於去除靜態偏差，並改善系統的穩定性，而微分分量則用於對抗過度響應和抑制震盪。

PID 控制器被廣泛應用於自動控制系統中的傑出控制算法，其可以通過計算誤差及其對整個系統的響應，實現自動的升溫或降溫控制，從而提高系統的穩定性和性能。

PID 控制器的過衝、反應時間、穩定時間和穩態誤差是衡量控制系統不同性能指標的重要參數。過衝越小，代表系統響應的品質越好，可以快速且精準的回到設定值；反應時間越短，系統越快速地達到設定值，但過於縮短則可能會導致系統不穩定；穩定時間越短，表示系統越快速地達到穩態。穩態誤差越小，表示系統的控制效果越好，越能夠達到期望的控制目的。

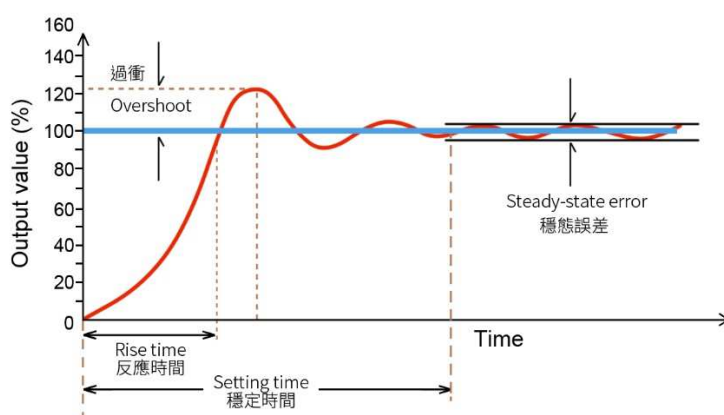


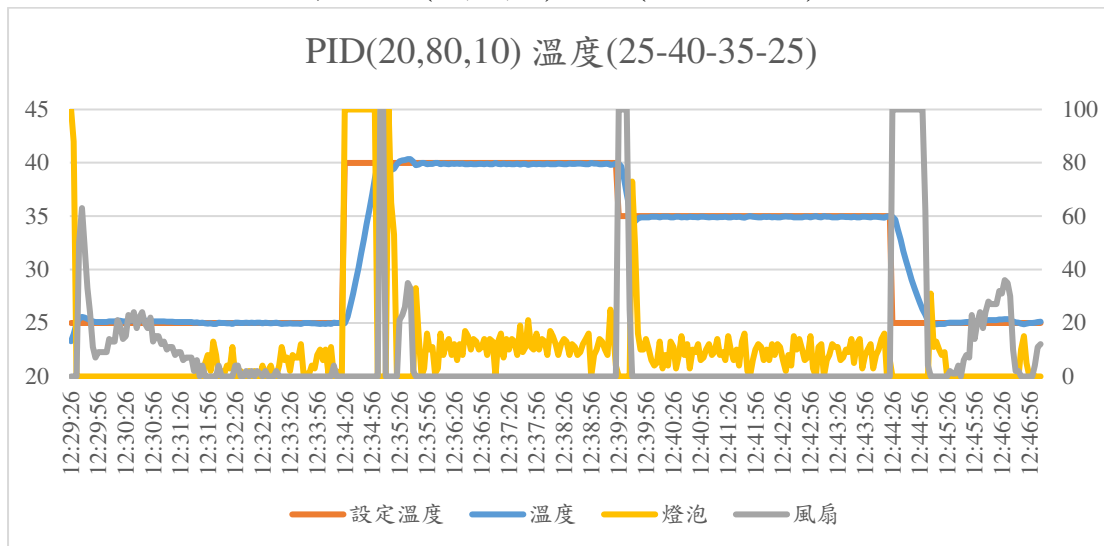
圖 18. PID 指標



圖 17. PID 程式節錄

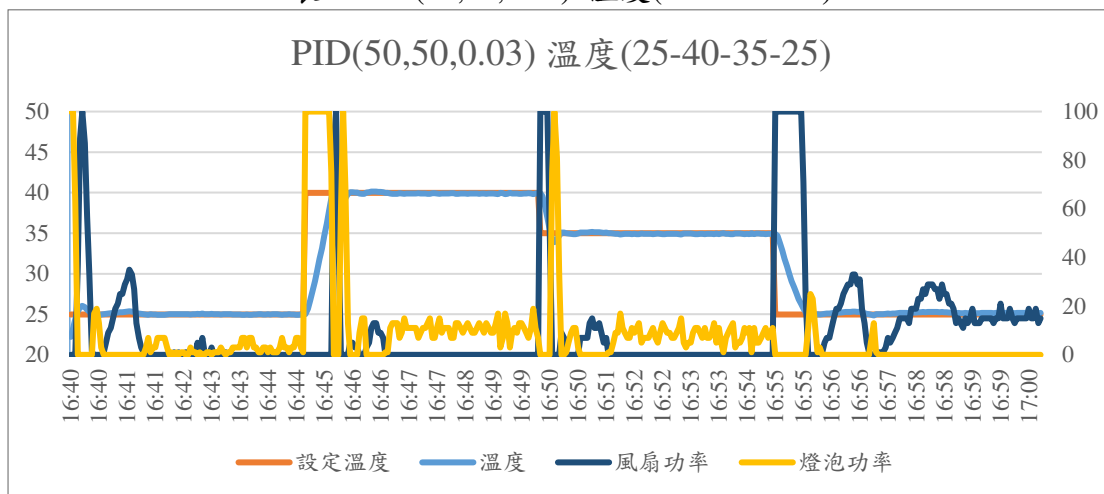
在不適合的 PID 數值中我們可以發現風扇跟燈泡的起伏都過大，也無法做到精確的控制。在溫度的誤差上可以測得有較大的誤差，以下是不合適的 PID 參數($K_p = 20, K_i = 80, K_d = 10$)，使用溫度變化(25 升溫 40 降溫 35 降溫 25)所測試繪出的圖表，可以看到他的燈泡跟風扇開啟幅度較大，這也是我們不樂見的，在細部上可以看到反應跟穩定時間也比我們調整過後的長許多。

表 1 PID(20,80,10) 溫度(25-40-35-25)



系統經過多次測試後，從設定溫度後開始到溫度到達所設定的溫度，速度極快，達到恆溫後的溫度上下震盪，誤差約在 0.15 度。在圖表中可以看到溫控箱在達到溫度後，大多是使用燈泡進行更精準的控溫使震盪在可控的範圍內，在下圖中我們一樣選用(25 升溫 40 降溫 35 降溫 25)，這個溫度變化，每五分鐘改變一次溫度，來觀測他的升溫降溫與震盪的變化。此 PID 參數($K_p = 50, K_i = 50, K_d = 0.03$)是我們嘗試多次調整後獲取的最佳 PID 數值。

表 2. PID(50,50,0.03) 溫度(25-40-35-25)



在下方兩張圖表中我們縮小了範圍，可以更清楚看到測試的兩組 PID 中的實驗數據，上方(50,50,0.03)組中雖然過衝的數值高了點，但比下方(20,80,10)組中有更好

的反應時間、安定時間，這樣對比可以讓我們知道，找到好的 PID 數值可以讓我們更好的控制各種設備。

表 3. PID(20,80,10) 溫度(25-40)

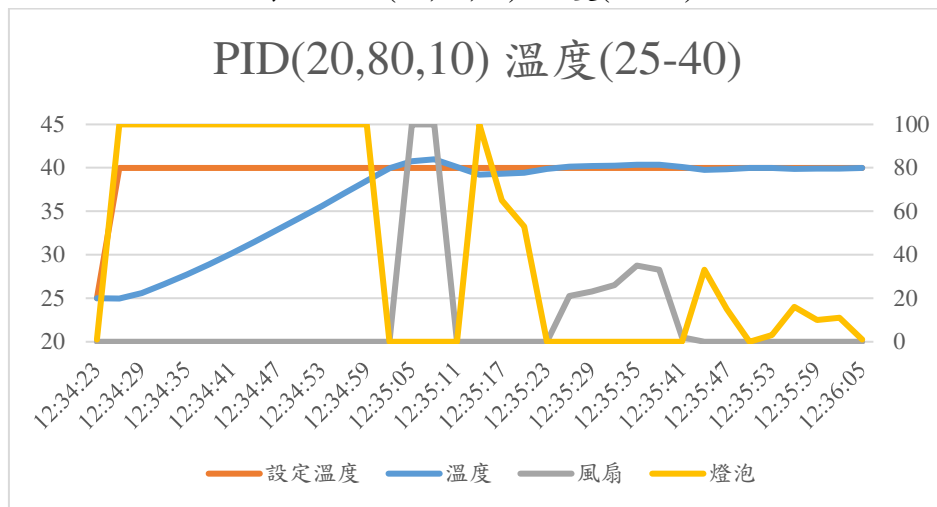
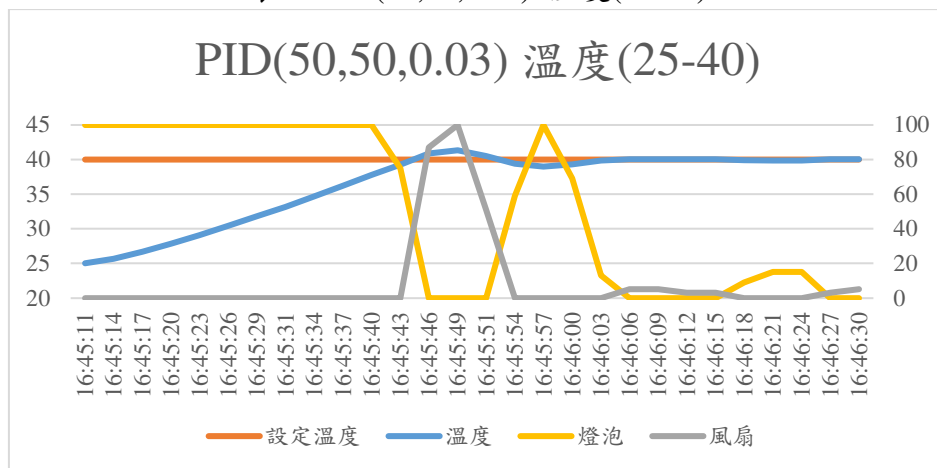
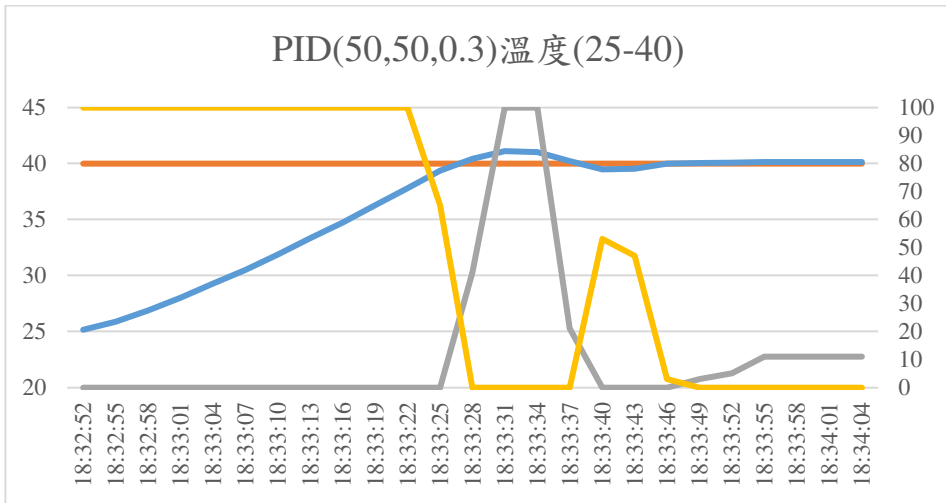


表 4. PID(50,50,0.03) 溫度(25-40)



我們再更進一步調整 Kd 的數值希望能降低過衝的數值，可以在下途中發現過衝的數值的確下降，但其他的數值沒有明顯變化，可以看到我們更進一步優化了溫度的過衝問題。

表 5. PID(50,50,0.3) 溫度(25-40)



在下表可以看出優化前優化後的差別，PID 的各項指標，在還沒有優化前我們隨意的下了一些參數可以看到他的安定時間並不理想，所以我們在此基礎上嘗試了多個參數，找到(50,50,0.03)這個數值將安定時間控制在一分鐘以內，但是出現較高的過衝，我們進一步調整 Kd 的數值將過衝再下降了 0.22°C，並維持了原本的反應時間與安定時間。

表 5.各參數對比

	反應時間	過衝	安定時間	穩態誤差
參數優化前 20,80,10 (25°C-40°C)	36 秒	0.97°C	84 秒	約 0.15°C
參數優化後 50,50,0.03 (25°C-40°C)	32 秒	1.33°C	53 秒	約 0.15°C
進一步優化 50,50,0.3 (25°C-40°C)	36 秒	1.11°C	54 秒	約 0.15°C

4.結論

本研究使用 PID 控制技術進行智能恆溫裝置的開發，並透過低成本的元件和 Arduino 開發晶片實現控制。其中，PID 演算法用於智能調節溫度，PWM 技術被用於控制燈泡和風扇。通過建置雲端伺服器和網頁顯示的方式，實現了完整的智能恆溫監控系統，使用者或維護人員可以透過網路進行環境溫度監控。

由於 PID 參數需要針對不同的情況進行調整，本研究透過雲端系統將所有數據儲存到資料庫中，以便進行數據分析，並且能個別優化不同情況下的 PID 參數，或者進行環境研究。除此之外，本研究使用 MQTT 和 NodeRed 建置系統，提高系統的維護效率，從而實現更穩定高精準度的恆溫控制。

使用 MQTT 與 NodeRed 搭建的物聯網系統實現了穩定與快速等優點，在進一步搭配區域網路的使用下，更能確保整套系統不會失去使用者的控制，使用 MQTT 與 NodeRed 也確保了擴展性，如果需要增加或減少控制的器具、物品也可以做到快速的疊代更新。

本研究的智能恆溫系統對於廣泛的應用場景都具有實用價值，實現了對環境溫度的智能監控和控制，並且整合了數據儲存、分析，從而實現了高效率、高穩定性的恆溫控制。

參考文獻

1. Dong, B., Shi, Q., Yang, Y., Wen, F., Zhang, Z., & Lee, C. (2021). Technology evolution from self-powered sensors to AIoT enabled smart homes. *Nano Energy*, 79, 105414. doi: 10.1016/j.nanoen.2020.105414
2. Aghaei, V. T., Onat, A., Eksin, I., & Guzelkaya, M. (2015, July). Fuzzy PID controller design using Q-learning algorithm with a manipulated reward function. In 2015 European control conference (ECC) (pp. 2502-2507). IEEE. doi: 10.1109/ECC.2015.7330914
3. Wang, C. (2009, November). A thermostat fuzzy control system based on MCU. In 2009 Third International Symposium on Intelligent Information Technology Application (Vol. 1, pp. 462-463). IEEE. doi: 10.1109/IITA.2009.324
4. Li, Z., Hu, J., & Huo, X. (2012, October). PID control based on RBF neural network for ship steering. In 2012 World Congress on Information and Communication Technologies (pp. 1076-1080). IEEE. doi: 10.1109/WICT.2012.6409235
5. Rodríguez, J. R., Dixon, J. W., Espinoza, J. R., Pontt, J., & Lezana, P. (2005). PWM regenerative rectifiers: State of the art. *IEEE Transactions on Industrial Electronics*, 52(1), 5-22. doi: 10.1109/TIE.2004.841149
6. Parihar, Y. S. (2019). Internet of things and nodemcu. *journal of emerging technologies and innovative research*, 6(6), 1085.
7. Jin, G., Zhang, X., Fan, W., Liu, Y., & He, P. (2015). Design of non-contact infra-red thermometer based on the sensor of MLX90614. *The Open Automation and Control Systems Journal*, 7(1). doi: 10.2174/1874444301507010006
8. Rusimamto, P. W., Endryansyah, L. A., Harimurti, R., & Anistyasari, Y. (2021). Implementation of arduino pro mini and ESP32 cam for temperature monitoring on automatic thermogun IoT-based. *Indones. J. Electr. Eng. Comput. Sci*, 23(3), 1366-1375. doi: 10.11591/ijeecs.v23.i3.pp1366-1375
9. Hunkeler, U., Truong, H. L., & Stanford-Clark, A. (2008, January). MQTT-S—A publish/subscribe protocol for Wireless Sensor Networks. In 2008 3rd International Conference on Communication Systems Software and Middleware and Workshops (COMSWARE'08) (pp. 791-798). IEEE. doi: 10.1109/COMSWA.2008.4554519
10. Kodali, R. K., & Anjum, A. (2018). IoT Based HOME AUTOMATION Using Node-RED. doi: 10.1109/ICGCIoT.2018.8753085
11. Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to build high-performance network programs. *IEEE Internet Computing*, 14(6), 80-83. doi: 10.1109/MIC.2010.145
12. 趙英傑. (2013). 超圖解 Arduino 互動設計入門. 旗標出版股份有限公司.
13. Cui, G., Xiao, P., & Li, Y. (2012). Based on the heating furnace temperature Fuzzy-PID control method research. <https://doi.org/10.1109/wcica.2012.6358125>
14. Åström, K. J., & Hägglund, T. (2001). The future of PID control. *Control engineering practice*, 9(11), 1163-1175. doi: 10.1016/S0967-0661(01)00062-4
15. Kodali, R. K., & Soratkal, S. (2016, December). MQTT based home automation system using ESP8266. In 2016 IEEE Region 10 Humanitarian Technology Conference (R10-HTC) (pp. 1-5). IEEE. doi: 10.1109/R10-HTC.2016.7906845
16. Mesquita, J., Guimarães, D., Pereira, C., Santos, F., & Almeida, L. (2018, September). Assessing the ESP8266 WiFi module for the Internet of Things. In 2018 IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA) (Vol. 1, pp. 784-791). IEEE. doi: 10.1109/ETFA.2018.8502562
17. Soni, D., & Makwana, A. (2017, April). A survey on mqtt: a protocol of internet of things (iot). In International conference on telecommunication, power analysis and

- computing techniques (ICTPACT-2017) (Vol. 20, pp. 173-177).
18. Karagiannis, V., Chatzimisios, P., Vazquez-Gallego, F., & Alonso-Zarate, J. (2015). A survey on application layer protocols for the internet of things. *Transaction on IoT and Cloud computing*, 3(1), 11-17.
 19. Kodali, R. K., & Soratkal, S. (2016, December). MQTT based home automation system using ESP8266. In 2016 IEEE Region 10 Humanitarian Technology Conference (R10-HTC) (pp. 1-5). IEEE. doi: 10.1109/R10-HTC.2016.7906845
 20. Ferencz, K., & Domokos, J. (2019). Using Node-RED platform in an industrial environment. XXXV. Jubileumi Kandó Konferencia, Budapest, 52-63.
 21. Lekić, M., & Gardašević, G. (2018, March). IoT sensor integration to Node-RED platform. In 2018 17th International Symposium Infoteh-Jahorina (Infoteh) (pp. 1-5). IEEE. doi: 10.1109/INFOTEH.2018.8345544